Figure A: Avalanche Validator Lifecycle Model v0.8.0 — Daily State Transition Overview

Macro Execution Architecture: Two cadCAD PSUBs, Complete State Feedback X_(t+1) → X_t, Separated Input Categories, and Invariant Verification

Complete Current State X_t

`X_t = (C_t, O_t, Z_t) ∈ X`

Active Commitments
`C_t (Micro-State)`
stakes, days_left, done, node_key

Out-of-Set Records
`O_t (Eligible Pool)`
out_stakes, out_days ≤ 365d

Global Protocol State
`Z_t = (S_t : supply_navax)`
Cumulative supply counter

X_t

Exogenous Inputs & Parameters U_t

Scheduled Commitments [Time-Varying]
`Pre-accepted queue due on day t`
Q_t (bypasses behavioural selection)

Protocol Configuration [Immutable Choice]
`Minting rate schedule & bounds`
ψ_prot : Bounds S_min=2k, S_max=3M; ACP-285

Historical Reference Data [Static Reference]
`Benchmark yield rates & quantiles`
D = (D_menu, continuation/stake tables)

Bootstrap Parameters [Run-Specific]
`Drawn once per Monte Carlo run`
θ_r = (β_r, γ_r, λ_0r, φ_r, δ_r)

Keyed PRNG Streams [Stochastic Draw]
`Common random numbers tied to keys`
ξ_(r,t) : Stateless keyed uniform streams

inputs U_t

PSUB 1: Main Daily Lifecycle Transition Block

cadCAD block returning `p_lifecycle` policy and 24 parallel state-update functions

Lifecycle Policy Function: p_lifecycle

Calls `run_stages(stages, state=X_t, day)` executing the 6 internal stages sequentially:

1. **Renewal** — renewal_stage (Logit)
2. **Returns** — returns_stage (Hazard)
3. **Arrivals** — arrivals_stage (NegBin)
4. **Continuation** — Duration + Stake Choice
5. **Rewards** — rewards_stage (Pricing)
6. **Assembly** — assembly_stage (Mutation)

Note: Duration selection and stake selection execute inside Stage 4 (_stake_continuation). Exogenous Q_t enters at Stage 6 (Assembly). Stages 1-6 are NOT independent PSUBs.

`COMMON POLICY SIGNAL (stakes, days, out_stakes, issued_navax...)`

State-Update Function Bank: Exactly 24 Functions

Evaluated in parallel on pre-PSUB state snapshot X_t and common policy signal:

1. `Commitment Columns s_copy(name)` [8 Functions]
Keys: stakes, days_left, done, after_return, days, handed_on, returns, node_key
Logic: Copies active commitment array slices for C_(t+1)

2. `Out-of-Set Columns s_copy(name)` [6 Functions]
Keys: out_stakes, out_days, out_done, out_last_days, out_returns, out_node_key
Logic: Copies eligible out-of-set array slices for O_(t+1)

3. `Global Supply Accumulation s_supply` [1 Functions]
Keys: supply_navax (Persistent Protocol State Z)
Logic: state["supply_navax"] + signal["issued_navax"] (Pre-PSUB S_t + flow)

4. `State Readout Metrics (s_count, s_stake, s_out, s_nakamoto)` [4 Functions]
Keys: active_validators, active_validator_stake_navax, returning_nodes, nakamoto_third_nodes
Logic: Aggregates: len(stakes), sum(stakes), len(out_stakes), Nakamoto key ratio

5. `Intermediate Flow Variables s_copy(name)` [5 Functions]
Keys: issued_navax, yield_gap_pp, started_stake_navax, started_stake_days_navax, offered_rate_30d
Logic: Copies daily fluxes and diagnostic signals into state record for metric readout

`Total State-Update Functions Reconciled: 8 + 6 + 1 + 4 + 5 = EXACTLY 24`

X_(t+1)'

PSUB 2: Invariant Validation

Policy **`p_invariants`** asserts 5 invariant assertions

Check 1: Validator Stake Bounds [Bounds]
`2,000 ≤ s_i ≤ 3,000,000 AVAX`
Evaluates candidate post-update state

Check 2: Stake Sum Conservation [Consistency]
`active_validator_stake_navax == sum(stakes)`
Evaluates candidate post-update state

Check 3: Out-of-Set Max Horizon [Retention]
`all(out_days ≤ 365)`
Evaluates candidate post-update state

Check 4: Protocol Supply Cap [Supply Limit]
`supply_navax ≤ 720,000,000 * 10^9 nAVAX`
Evaluates candidate post-update state

Check 5: Solvency Constraint [Solvency]
`active_validator_stake_navax ≤ supply_navax`
Evaluates candidate post-update state

Do All 5 Assertions Hold?

False → **FAILURE** `InvariantError` Halts run

True (Pass)

Validated Next State
`X_(t+1) (Sound State)`
C_(t+1), O_(t+1), Z_(t+1)

Recorded Outputs & Metrics

`Y_t = H(X_(t+1), Φ_t)`

`active_validators` [Snapshot]
N_(t+1) = |C_(t+1)|
Count of active commitment records

`active_validator_stake_navax` [Snapshot]
∑_(i ∈ C) s_(i,t+1)
Total active stake on Primary Network

`returning_nodes` [Snapshot]
M_(t+1) = |O_(t+1)|
Count of eligible out-of-set records

`nakamoto_third_nodes` [Snapshot]
min {m : ∑ S_(j) > 1/3 S_tot}
Decentralization over simulated node keys

`supply_navax` [Cumulative]
S_(t+1) = S_t + I_t
Persistent reward-adjusted supply

`issued_navax` [Flow]
I_t = ∑_(m ∈ N) I_(m,t)
Potential rewards from new endogenous starts

`started_stake_navax` [Flow]
∑_(m ∈ N) s'_m
Principal stake committed today

`started_stake_days_navax` [Flow]
∑_(m ∈ N) s'_m * d'_m
Contracted stake-duration flux

`yield_gap_pp` [Signal]
g_t^(R) = offered - benchmark
Renewal yield gap in percentage points

`offered_rate_30d` [Diagnostic]
rate(30d ; S_t, ψ_prot)
Annualized rate offered for baseline stake

COMPLETE STATE FEEDBACK: Validated X_(t+1) = (C_(t+1), O_(t+1), Z_(t+1)) becomes Input X_t for Day t+1

Visual Grammar & Adapted Notation Legend

- **Cylinder (Blue-Grey):** Persistent Endogenous State Store (C_t, O_t, Z_t)
- **Inverted Triangle (Cream):** Exogenous Input / Scenario Parameter (Q_t, D, ψ_prot)
- **Diamond (Muted Pink):** Stochastic Behavioural Policy / Decision Stage
- **Pentagon (Sage Green):** Mechanism / State Update Function *
- **Double-Border Box (Red):** Formal Invariant Assertion (PSUB 2; raises InvariantError on failure)
- **Rectangle (Grey):** Recorded Output Metric / Observable Vector Y_t
- **Solid Arrow:** State / Signal Dataflow
- **Dashed Blue Line:** Inter-Timestep Feedback Loop (X_(t+1) → X_

** Adapted Visual Convention: In complex systems engineering (e.g. BlockScience / Polkadot specifications), pentagons denote control functions. Here, pentagons denote deterministic protocol mechanisms and state-update operations.*

Model Boundaries: S_min = 2,000 AVAX constant (no duration-dependent admission frontier enforced in v0.8.0); no uptime/liveness/slashing; concentration over simulated node keys.